



计算机科学

Computer Science

ISSN 1002-137X, CN 50-1075/TP

《计算机科学》网络首发论文

题目：基于对比学习的双通道源代码漏洞检测模型
作者：宋建华，何佳伟，张龔
网络首发日期：2025-06-17
引用格式：宋建华，何佳伟，张龔. 基于对比学习的双通道源代码漏洞检测模型[J/OL]. 计算机科学. <https://link.cnki.net/urlid/50.1075.TP.20250616.1043.010>



网络首发：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

出版确认：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

基于对比学习的双通道源代码漏洞检测模型

宋建华^{1,3,4,5} 何佳伟¹ 张龔^{2,3,5}

- 1 湖北大学网络空间安全学院 武汉 430062
2 湖北大学计算机学院 武汉 430062
3 智能感知系统与安全教育部重点实验室 武汉 430062
4 智能网联汽车网络安全湖北省工程研究中心 武汉 430062
5 大数据智能分析与行业应用湖北省重点实验室 武汉 430062
(sjhhu@126.com)

摘要 随着软件漏洞日益增多, 系统安全正面临着严峻的挑战。源代码漏洞检测可以在软件开发阶段及时发现软件应用中的潜在安全威胁, 对于保障软件应用的安全性至关重要。目前, 主流的源代码漏洞检测方式为基于深度学习模型的漏洞检测方式。然而, 现有的许多深度学习模型仅依赖单一形式特征, 未能充分挖掘源代码语义中的全局和局部信息; 其次, 这些模型往往忽略了不同样本之间的差异性和相似性, 导致其在处理复杂漏洞模式时表现不佳, 误报率和漏报率较高。为了解决上述问题, 提出了一种基于对比学习的双通道源代码漏洞检测模型。该模型使用不同通道来分别提取源代码语义中的全局特征和局部特征, 并引入对比学习, 使得模型能够学习不同样本之间的相似性和差异性, 并以此来优化特征提取过程。实验结果表明, 此模型在真实世界的漏洞数据集 Devign 和 Reveal 上的召回率、F1 分数相较于基线模型显著提升。在 Devign 上平均提升 14.65 和 6.30 个百分点; 在 Reveal 上平均提升 31.18 和 22.44 个百分点。

关键词: 源代码漏洞检测; 双通道网络模型; 对比学习; 交叉注意力; 特征融合

中图分类号 TP311

Dual-Channel Source Code Vulnerability Detection Model Based on Contrastive Learning

Jianhua Song^{1,3,4,5}, Jiawei He¹, Yan Zhang^{2,3,5}

- 1 School of Cyber Science and Technology, Hubei University, Wuhan 430062, China
2 School of Computer Science, Hubei University, Wuhan 430062, China
3 Key Laboratory of Intelligent Sensing System and Security (Hubei University), Ministry of Education, Wuhan 430062, China
4 Hubei Provincial Engineering Research Center of Intelligent Connected Vehicle Network Security, Wuhan 430062, China
5 Hubei Key Laboratory of Big Data Intelligent Analysis and Application, Hubei University, Wuhan 430062

Abstract As software vulnerabilities continue to increase, system security is facing severe challenges. Source code vulnerability detection can identify potential security threats in software applications during the development phase, which is crucial for ensuring the security of software applications. Currently, the mainstream method for source code vulnerability detection is based on deep learning models. However, many existing deep learning models rely only on a single form of features and fail to fully explore both the global and local information in the source code semantics. Additionally, these models often overlook the differences and similarities between different samples, leading to poor performance in handling complex vulnerability patterns, with high false positive and false negative rates. To address these issues, a dual-channel source code vulnerability detection model based on contrastive learning is proposed. The model uses different channels to separately extract global and local features from the source code semantics, and introduces contrastive learning to enable the model to learn the similarity and difference between different samples, thereby optimizing the feature extraction process. Experimental results show that this model significantly improves the recall rate and F1 score compared to the baseline model on real-world vulnerability datasets Devign and Reveal. On Devign, it improves by an average of 14.65 and 6.30 percentage points; on Reveal, it improves by an average of 31.18 and 22.44 percentage points.

基金项目: 国家自然科学基金 (No. 62377009); 湖北省重大攻关项目 (JD) (No. 2023BAA018); 湖北省重点研发计划重点项目 (No. 2021BAA184, No. 2021BAA188); 湖北省高等学校人文社会科学重点研究基地绩效评价信息管理研究中心课题 (No. 2020JX01); 湖北省科技计划重大科技专项 (No. 2024BAA008)
This work was supported by the National Natural Science Foundation of China (No. 62377009); Major Project of Hubei Province (JD) (No. 2023BAA018); Key Project of Hubei Provincial Key R&D Program (No. 2021BAA184, No. 2021BAA188); Research Center for Performance Evaluation and Information Management of Key Research Bases for Humanities and Social Sciences in Hubei Provincial Colleges and Universities (No. 2020JX01); and Major Science and Technology Special Project of Hubei Science and Technology Plan (No. 2024BAA008).

通信作者: 何佳伟 (1766962638@qq.com)

leading to poor performance when handling complex vulnerability patterns, with high false positive and false negative rates. To address these issues, a dual-channel source code vulnerability detection model based on contrastive learning is proposed. This model uses different channels to separately extract global and local features from the source code semantics and introduces contrastive learning to allow the model to learn the similarities and differences between different samples, thereby optimizing the feature extraction process. Experimental results show that This model shows significant improvements in recall and F1 score on the real-world vulnerability datasets, Devign and Reveal, compared to the baseline model. The average improvement is 14.65 and 6.30 percentage points on Devign; and 31.18 and 22.44 percentage points on Reveal.

Keywords Source code vulnerability detection, Dual channel network model, Comparative learning, Cross attention, Feature fusion

1. 引言

在当今数字化时代，各类软件面临着严重的漏洞问题，漏洞数量呈现爆炸式增长，其危害性也日益加剧，安全漏洞的存在为恶意攻击提供了可乘之机，严重威胁着系统的安全性和数据的完整性，甚至可能引发许多社会性问题。根据 Skybox 在 2023 年发布的漏洞与趋势报告^[1]，国家漏洞数据库（NVD）在 2022 年新增了 25,906 个漏洞，相较于 2021 年增加了 25%。更为严重的是，其中超过 80% 的漏洞被评定为中高危漏洞。软件安全问题已成为全球亟待解决的重大挑战之一。

软件漏洞是指在软件开发或运行过程中因设计或实现缺陷导致的安全问题。早期发现并消除这些漏洞，特别是在代码编写阶段，能够有效降低系统的安全风险。因此，源代码漏洞检测技术成为确保软件安全性的关键手段之一。

根据检测方法的不同，软件漏洞检测可以划分为静态检测和动态检测两种方式^[2]。相较于动态检测，静态检测不依赖于程序执行，可以在源代码层面全面分析所有代码路径，且不需要额外的运行时资源，具有较高的效率和成本优势。

静态检测方法又可以根据所用技术的不同进一步分为基于规则的漏洞检测方法、基于传统机器学习的漏洞检测方法和基于深度学习的漏洞检测方法^[2]。相较于前两种方法，基于深度学习的漏洞检测方法通过自动化学习特征，能够从源代码

中提取复杂的语义特征，具备比传统机器学习方法更强的表达和建模能力，能够高效地处理大规模数据，同时具有较强的泛化能力以适应不同编程语言和项目环境。

近年来，基于深度学习的检测方法已成为源代码漏洞检测领域的主流技术，并取得了显著的进展^[3]。然而，常规的深度学习模型仍面临以下几个问题：首先，许多模型仅依赖单一特征提取形式，未能充分挖掘源代码语义中的全局信息和局部信息，从而导致特征提取得不全面，影响模型的检测性能；其次，大多模型往往忽略了不同检测样本之间的相似性和差异性，将样本视作孤立的单位作为模型的输入，进而影响了模型对各类漏洞的精确识别。为了解决这些问题，本文提出了一种基于对比学习^[4]的双通道源代码漏洞检测模型。一方面，使用两个通道分别提取全局特征和局部特征并进行融合，以此提高本文模型对源代码语义特征的提取能力；另一方面，通过引入对比学习，使本文模型能够学习不同检测样本之间的相似性和差异性，在特征空间中形成更精细的区分，从而进一步增强了漏洞检测的精度和鲁棒性。

综上所述，本文主要做出了以下贡献：

(1) 将不同的源代码样本两两视作一个整体，并引入对比学习来学习漏洞与漏洞样本、漏洞与非漏洞样本之间的潜在联系。

(2) 提出了一种双通道特征提取方式，分别

提取样本语义的全局特征和局部特征后,使用交叉注意力机制^[5]进行特征融合,最后输入至全连接层完成漏洞检测任务。

(3) 设计并实现了漏洞检测模型 MC-CLVD,通过实验验证了其检测性能,同时通过消融实验评估了模型中各个组成部分的贡献与作用。

2. 相关工作

2.1 源代码漏洞检测

深度学习能够从代码中自动提取语法、语义和结构特征,大幅减少对人工经验的依赖,随着越来越多的预训练模型(如 CodeBERT^[6]、RoBERTa^[7]、GraphCodeBERT^[8]等)的推出,深度学习在漏洞检测领域的应用愈发广泛。当前,基于深度学习的漏洞检测方法主要分为两类:基于 Token 序列表征的检测方法和基于图表征的检测方法。

基于 Token 序列表征的检测方法核心在于将源代码解析为保留了代码结构和上下文信息的 Token 序列,并进一步转换为适合神经网络处理的词向量表示。在这一领域,Russell 等人^[9]提出了一种方法,将程序源代码提取为 Token 序列后,转换为固定长度的向量表示,利用 CNN 和 RNN 提取特征,并将这些特征输入随机森林分类器以检测漏洞。随后,Li 等人^[10]提出了 VulDeePecker,该方法通过提取代码片段(code gadgets)来捕捉漏洞代码中的数据流和结构特征。Hanif 等人^[11]利用掩码语言建模(Masked Language Modeling, MLM)^[12]技术对 RoBERTa 模型进行训练,构建出预训练模型 VulBERTa,该模型能够生成更精确的代码表示,显著增强了模型对漏洞检测的理解能力和检测性能。但是,上述现有的工作在处理源代码样本时,大多使用单一通道对单一形式的特征进行处理,未能有效地结合源代码样本语义中的全局特征和局部特征,并且只关注了单个样本中的漏洞语义,忽略了漏洞样本和非漏洞样本之间的差异性和不同漏洞样本之间的相似性,使得模型在漏洞检测方面

仍然具有局限性。

基于图表征的检测方法能够结合数据流、控制流等特性,全面地表示出更加复杂的代码结构。Zhou 等人^[13]提出的 Devign,将自然代码语言序列与 AST、CFG、DFG 结合起来构建源代码的图表示,并且设计了一个新的卷积模块来进行特征提取。Nguyen 等人^[14]提出的 ReGVD 方法,将源代码表示为扁平化的 Token 序列并构建图表示,利用 GNN 进行特征提取。Ling 等人^[15]提出了一个由 GCN 和 GAT 组成的双图神经网络模块 DGVD,利用两个不同的图神经网络来获取控制流的全局结构信息和节点之间的关系,并通过基于卷积的特征增强模块来捕获不同尺度的信息。尽管基于图表征的检测方法在捕捉代码的全局结构和依赖关系方面具有明显优势。但是,这类方法需要提前对代码进行解析并构建图,这一过程较为繁琐复杂,尤其是对于大规模源代码库来说。并且图神经网络处理的是节点之间的连接关系和全局信息,可能无法细致地捕捉代码中的每一个 Token 语法、上下文语义和具体行为。

2.2 对比学习

对比学习是一种无监督学习的方法,最初被应用于学习视觉表示。其核心思想在于拉近正样本对的距离,使语义相似的样本在表征空间中距离更近;同时拉远负样本对的距离,使语义不同的样本在表征空间中距离更远。在自然语言处理领域,Jain 等人^[16]提出了一种对比代码表示学习方法来解决 RoBERTa 对源代码编辑的敏感性问题,以获得更稳健的代码表示。Neelakantan 等人^[17]使用对比学习目标训练嵌入模型,在未标记的数据集上来生成高质量的向量表示。Liu 等人^[18]提出的 ContraBERT 在程序和自然语言序列上设计了一组增强算子以获得不同变体,将变体与对比学习相结合后用于克隆检测、漏洞检测等任务。Chen 等人^[19]提出的 Clear 将对比学习应用于智能合约漏洞检测领域,通过挖掘不同类型样本间的共性,有效提升模型的泛化能力。然而,上述工作也存在未能有效地结合源代码样本语义中

的全局特征和局部特征的问题。

针对上述工作的不足, 本文提出的 MC-CLVD 模型采取不同通道来分别提取样本语义中的全局特征和局部特征。同时, 为了更好地提取代码中的语义特征, 本文在 MC-CLVD 模型中引入对比学习机制, 以增强模型在处理漏洞样本与非漏洞样本之间的差异以及不同漏洞样本之间相似性的学习能力。

3. 基于对比学习的双通道源代码漏洞检测模型

本章主要介绍 MC-CLVD 模型的整体设计逻辑, 以及不同模块的功能实现。该模型由数据预处理、基于双通道的特征提取和基于特征融合的漏洞检测三个部分组成, 整体模型示意如图 1 所示。

3.1 数据预处理

数据预处理是自然语言处理任务中的关键步

骤, 旨在提高原始数据的质量和可用性。具体而言, 本节主要包括注释清理、词嵌入和上下文语义表征学习三个部分。

3.1.1 注释清理

在将数据转换为词嵌入表示之前, 需要对源代码进行预处理, 以消除可能对模型学习产生不利影响的干扰因素。通过统计分析发现, 代码样本中约有一半包含注释, 但其中不少注释内容冗长或无意义, 可能会增加噪声并干扰模型对关键语义特征的聚焦。

为了解决这一问题, 本文采用基于正则表达式的匹配方法自动清除代码中的注释内容以及多余的换行符和制表符。这种方法不仅能够有效地减少代码输入的长度, 还能够在后续处理环节中降低模型对无关信息的依赖, 减少噪声对模型训练的干扰。

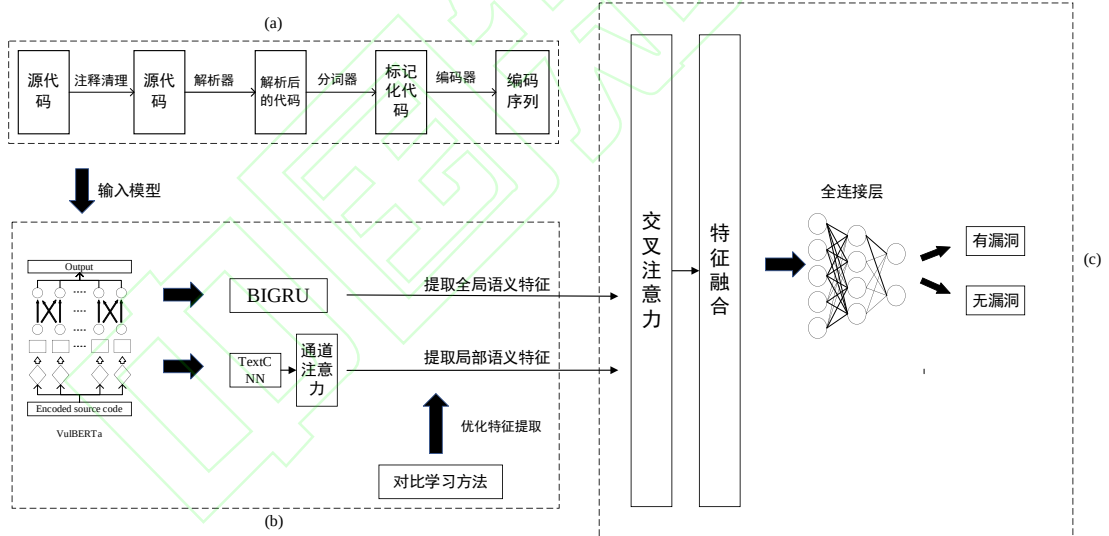


图 1 MC-CLVD 整体模型图。(a) 表示数据的预处理过程; (b) 表示基于对比学习的特征提取过程; (c) 表示基于特征融合进行漏洞检测

Fig 1. The overall model of MC-CLVD. (a) Represents the data preprocessing process; (b) Represents the multi-channel feature extraction process based on contrastive learning; (c) Represents vulnerability detection based on feature fusion.

3.1.2 词嵌入

本文中的词嵌入过程由解析器、分词器和编码器三个不同组件共同实现, 每个组件在将词汇转化为向量表示中扮演着重要角色。

1) 解析器

本文使用 C/C++ 解析工具 Clang^[20]对注释清理后的源代码进行解析。Clang 对代码进行词法分析, 在保留源代码句法结构的同时将源代码分解为一系列 Token, 这些 Token 是代码的基本组成单位, 包含了关键字、标识符、操作符和变量

等。

2) 分词器

解析生成的 Token 会进一步交由 BPE (Byte Pair Encoding) [21]子词分词算法处理, 将其转换为子词单元。BPE 是一种基于频率的分词方法, 通过合并高频字符或子词对逐步生成较优的分词结果。为了保证 C/C++中的预定义关键字不受分词影响 (如 int、void、if 等), 本文选择将这些关键字排除在 BPE 处理之外。同时, 为提升分词器的鲁棒性, 新增了几个特殊 Token 来应对不同情况, 例如<unk>用于表示未知字符。

3) 编码器

编码是将 Token 转换为向量的关键过程。通过对本文所使用的数据集中的源代码样本进行评估, 本文选择将最大序列长度设置为 1024, 以覆盖 90%的输入样本。对于超出最大序列长度的

部分, 采用截断处理; 而对于不足最大序列长度的部分, 则使用<pad>进行填充, 并设置相应的<mask>来区分填充区域, 以确保模型能够正确识别和处理输入数据的有效部分。数据处理过程如图 2 所示。

3.1.3 上下文语义表征学习

本文使用预训练模型 VulBERTa 来完成对源代码上下文语义特征的学习。该模型是基于 RoBERTa 架构的一种语言模型, 继承了其高效的 Transformer 编码器和强大的语义建模能力。针对代码的上下文语义表征需求, VulBERTa 采用了掩码语言技术作为预训练任务, 从而深度捕获源代码的上下文信息。该模型的预训练基于 Draper 和 GitHub 数据集, 这些数据集汇集了海量真实代码样本, 覆盖了多种编程语言及其上下文模式, 为模型学习提供了丰富的语料支持。

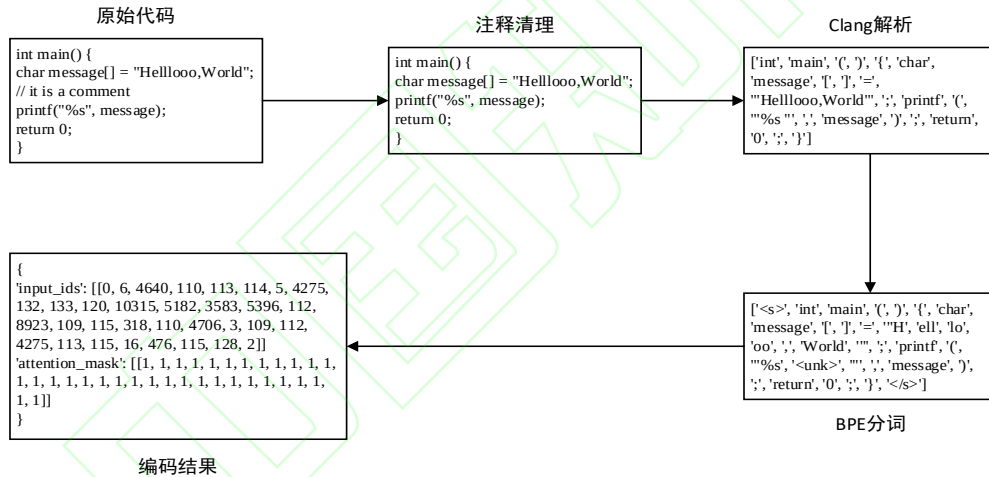


图 2 数据处理过程

Fig 2. Data processing process

3.2 基于对比学习的双通道特征提取

在完成数据预处理后, 接下来的任务是如何从数据中提取出语义特征。本节将详细介绍这一过程, 包括语义特征的分析、使用不同通道来分别提取样本语义中的全局特征和局部特征和如何利用对比学习来优化特征提取过程。

3.2.1 全局特征和局部特征分析

全局特征通常涉及跨多个函数、模块或类的代码特征。这些特征描述了代码的整体结构和执行路径, 能揭示出跨函数或模块的潜在漏洞。全

局特征一般包括控制流、数据流、函数间的调用关系等。局部特征则通常聚焦于单个函数或代码块中的行为, 更加关注局部的变量操作、内存分配等。

以下通过一个存在缓冲区溢出漏洞的代码示例来说明源代码语义中全局特征和局部特征。代码示例如图 3 所示。

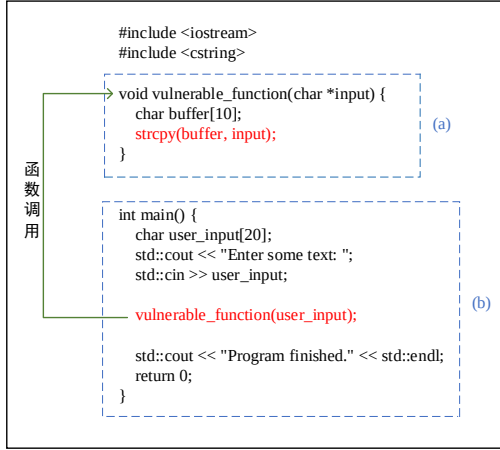


图 3 代码示例。其中用红色标注的代码表示漏洞触发点；虚线框中的 (a) 和 (b) 属于不同的代码块

Fig 3 Code example. The code marked in red represents the vulnerability trigger point; The (a) and (b) in the dashed box belong to different code blocks

在示例中，跨函数的数据传递和控制流是全局特征的关键。`main` 函数决定了程序的执行路径，特别是用户输入的数据如何从 `main` 函数传递到 `vulnerable_function` 函数中。这种跨函数的数据传递影响了程序的行为，导致了缓冲区溢出漏洞的发生。

局部特征主要体现在 `vulnerable_function` 函数内部，特别是字符串赋值操作。`strcpy(buffer, input)` 直接将 `input` 复制到 `buffer` 数组中，没有检查其长度，若 `input` 的长度超过了 `buffer` 的大小，就会发生缓冲区溢出。

综上所述，全局特征帮助漏洞检测系统理解程序中多个部分如何协作。当涉及跨多个函数、代码块的逻辑错误时，全局特征对于发现这些问题至关重要；局部特征则帮助检测函数内部的错误，特别是与资源管理和边界条件相关的问题，如内存泄漏、数组越界等。通过检查函数内部的操作，模型可以发现这些具体的错误。

3.2.2 全局特征提取模块

在处理源代码漏洞检测任务时，由于代码的层次性和依赖性，其中的结构化信息如函数调用、数据流向等往往需要考虑代码的上下文信息才能理解其语义。因此，源代码中的漏洞往往不局限于单一函数或代码片段，而是涉及跨多个代码块、函数乃至模块的复杂交互关系。传统的单

向递归神经网络 (RNN) 和门控循环单元 (GRU) 由于梯度消失问题或单向结构的限制，难以有效捕捉这些长距离依赖关系，它们只能依赖历史状态，缺乏对未来上下文的考量，这使得它们在提取全局特征时面临局限。

双向门控循环单元 (BiGRU) 使用更新门和重置门来控制信息流动，解决了传统 RNN 梯度消失的问题。并将正向和反向的 GRU 网络结合起来，同时捕捉序列中当前时刻及其之前的上下文信息和当前时刻及其之后的上下文信息，如公式 (1) 和公式 (2) 所示：

$$\vec{h}_t = \sigma(\vec{W}x_t + \vec{U}\vec{h}_{t-1} + \vec{b}) \quad (1)$$

$$\vec{h}_t = \sigma(\vec{W}x_t + \vec{U}\vec{h}_{t+1} + \vec{b}) \quad (2)$$

其中， $\vec{h}_t$ 和 $\vec{h}_t$ 分别表示 t 时刻正向 GRU 层和反向 GRU 层的隐藏状态， x_t 表示 t 时刻的输入， $\vec{W}$ 和 $\vec{U}$ 表示正向计算中的权重矩阵， $\vec{W}$ 和 $\vec{U}$ 表示反向计算中的权重矩阵， $\vec{b}$ 和 $\vec{b}$ 分别表示正向计算和反向计算中的偏置， σ 表示非线性激活函数。

模型时间步 t 的输出 y 由正向 GRU 层和反向 GRU 层的输出拼接得出，如公式 (3) 所示：

$$y_t = [\vec{h}_t, \vec{h}_t] \quad (3)$$

这种双向的信息流动使得 BiGRU 能够全面理解整个序列中的上下文，尤其是当代码中的逻辑或数据流跨越多个函数或代码模块时，BiGRU 能够同时从前后两个方向获取信息，更全面地理解源代码中跨函数调用的依赖关系，进而提取出全局特征。此外，本文使用预训练模型 VulBERTa 最后一层隐藏层的输出作为 BiGRU 的输入，将 BiGRU 与 VulBERTa 的深层语义表征结合，使得 BiGRU 能够在预训练模型提供的高阶语义基础上提取信息，进一步增强了 BiGRU 的全局特征提取能力。

为了优化模型性能，本文采用交叉熵损失函数作为目标函数，旨在最小化预测误差。交叉熵损失函数常用于分类任务中，通过衡量模型预测的概率分布与实际标签分布之间的差异，优化模型参数，目标是缩小两者的差距，使模型尽可能

接近真实标签。对于本文研究的源代码漏洞检测任务，计算公式如(4)所示：

$$loss_{cls} = -(y \log(p) + (1 - y) \log(1 - p)) \quad (4)$$

其中 y 是真实标签，取值为 0 或 1， p 是模型预测的类别为 1 的概率。

3.2.3 局部特征提取模块

在源代码漏洞检测任务中，虽然全局特征能够有效地描述输入序列的宏观结构，但常难以捕捉到序列中细微的局部信息和短期依赖关系。因此，本文选择进一步提取局部特征，以此来提高模型的鲁棒性，使其能够更好地适应不同尺度的信息，从而提升模型在复杂任务中的性能。

通常，卷积神经网络可以用来提取局部特征。TextCNN 是一种针对文本数据的卷积神经网络模型，本文使用三层一维卷积层（卷积核数量分别为 3、4、5）来提取文本的 n -grams 局部特征，卷积操作如公式(5)所示：

$$conv_k(x) = ReLU(Conv1d(x, W_k)) \quad (5)$$

其中， x 是输入的文本特征即嵌入后的词向量，Conv1d 表示 1 维卷积操作， W_k 表示卷积核。

为了增强 TextCNN 模型的性能，受到 Hu 等人^[22]提出的 SE (Squeeze-and-Excitation) 模块的启发，本文将通道注意力机制应用于文本数据的处理上。该机制的核心是通过动态地学习卷积层中每个通道的重要性，调整通道的权重，从而增强模型对关键信息的关注。在 TextCNN 的每个卷

积层输出之后，通过全局平均池化对每个卷积特征图进行压缩，得到一个标量表示该通道的全局信息，如公式(6)所示：

$$z_c = \frac{1}{T} \sum_{t=1}^T \hat{x}_{ct} \quad (6)$$

其中， $\hat{x}_{ct}$ 表示卷积特征图在时间步 t 和通道 c 的特征值， T 表示序列的长度， z_c 表示该通道的整体信息。

然后通过全连接层和 Sigmoid 激活函数计算每个通道的重要性权重 s_c ，如公式(7)所示：

$$s_c = \sigma(W_2 ReLU(W_1 z_c)) \quad (7)$$

其中， W_1 和 W_2 表示全连接层的权重矩阵。最后，将通道的重要性权重和卷积层的输出特征层相乘，实现通道特征的加权，如公式(8)所示：

$$\hat{x}_c = s_c \times x_c \quad (8)$$

其中， x_c 是卷积操作后第 c 通道的输出。最后，加权后的特征经过池化层和拼接处理后，形成最终的特征向量。过程如图 4 所示。

这种方式结合了卷积神经网络的局部特征提取能力和通道注意力机制的自适应加权能力，使得模型能够更加专注于有价值的特征，有效提升了模型的性能。

局部特征提取模块的损失函数选取与全局特征提取模块一致，均为交叉熵损失函数，故不再赘述。

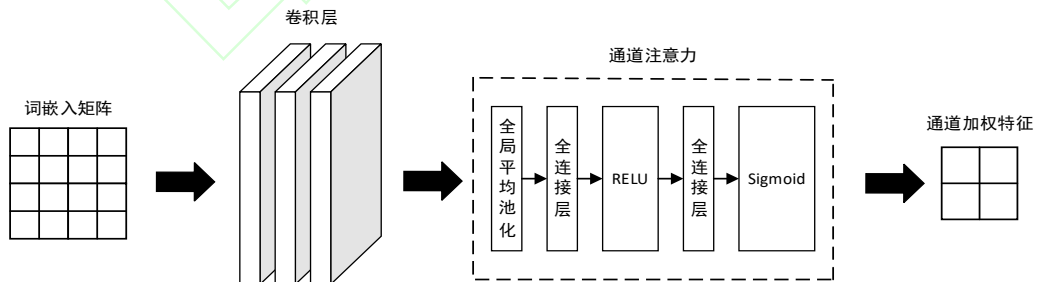


图 4 基于通道注意力机制的 TextCNN

Fig 4. TextCNN based on channel attention mechanism

3.2.4 对比学习方法

传统的源代码漏洞检测任务通常将每个源

代码样本视为孤立输入，忽略了漏洞样本（V）之间以及漏洞样本与非漏洞样本（N）之间的潜

在联系。为弥补这一不足,本文设计了一种基于对比学习的特征提取方法。该方法将样本对作为原始输入,样本对分为两种类型:漏洞样本与漏洞样本的配对(V-V),以及漏洞样本与非漏洞样本的配对(V-N)。具体采样策略如下:首先,从训练集中提取出所有漏洞样本,构成漏洞样本集合 Set;然后,对于原始训练集中的每个样本,从漏洞集合 Set 中随机选取一个漏洞样本与其配对,生成 V-V 对或者 V-N 对。标签的定义如公式(9)所示。此策略还有效避免了 N-N 对的生成,确保样本对在对比学习中的有效性。

$$\text{target}_{CL} = \begin{cases} 1, & \text{V-V} \\ 0, & \text{V-N} \end{cases} \quad (9)$$

其中 target_{CL} 表示样本对的标签;V-V表示样本对由两个漏洞样本组成;V-N表示样本对由一个漏洞样本和一个非漏洞样本组成。通过学习漏洞-漏洞样本和漏洞-非漏洞样本之间的潜在关联,能够为后续的特征提取任务提供更优质的特征表达。

本文采用余弦相似度损失函数作为对比学习的优化目标。该损失函数通过度量向量之间的方向相似性来优化模型性能,其核心思想是通过衡量两个向量之间的余弦相似度,来评估它们在嵌入空间的语义相关性,余弦相似度如公式(10)所示:

$$\text{CosineSimilarity}(u, v) = \frac{u \cdot v}{\|u\| \|v\|} \quad (10)$$

余弦相似度的取值范围为 $[-1, 1]$,值越接近1,表示两个向量方向越相似;值越接近-1,表示两个向量方向表示越相反。为了实现相似样本对的余弦相似度最大化,以及非相似样本对的余弦相似度最小化,本文先通过公式 $1 - \text{CosineSimilarity}(u, v)$ 将余弦相似度转换为余弦距离,然后通过最小化余弦距离的形式来优化模型,最终的损失函数 loss_{cl} 如公式(11)所示:

$$\text{loss}_{cl} = \frac{1}{N} \sum_{i=1}^N [y_i \cdot (d_i)^2 + (1 - y_i) \cdot (\max(0, d_i - \text{margin}))^2] \quad (11)$$

其中 y_i 表示样本 i 的标签, d_i 表示样本 i 的余弦距离, margin 表示负样本的距离阈值,用来控制负样本之间的最小距离。

对比学习任务与分类任务(是否存在漏洞)共享用于特征提取的神经网络模型及相关参数,采取交替训练的形式,首先进行对比学习任务,通过这一过程,能够更好地捕捉样本之间的潜在联系,为分类任务提供更加精细和更具辨别力的特征表示;接着进行分类任务以完成特征提取,具体过程如图5所示。通过多任务协同训练,本文的模型不仅能够分类任务中表现地更好,还能够提高对复杂语义关系的理解和区分能力。

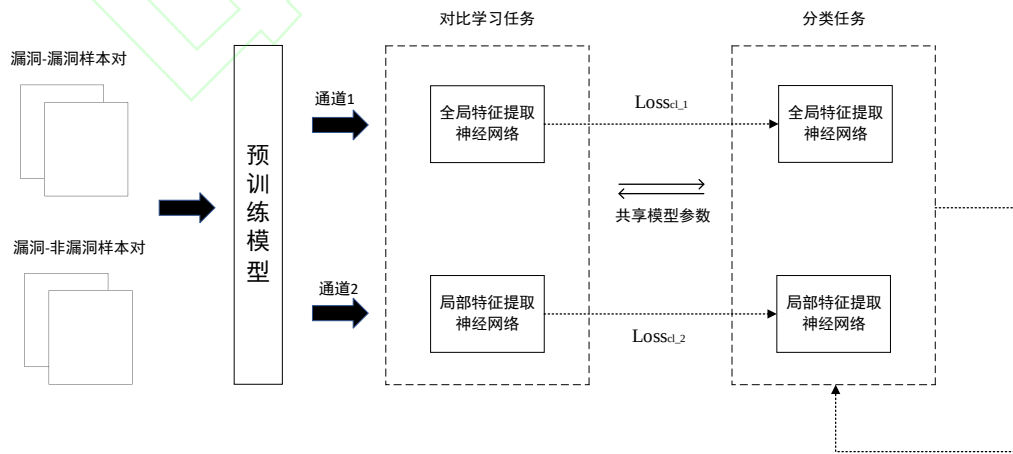


图 5 对比学习过程

Fig 5. Contrastive learning process

3.3 基于特征融合的漏洞检测

成功提取代码语义的全局特征和局部特征之后，模型就可以捕捉到输入数据中不同层面的信息。全局特征帮助抓取整体模式，而局部特征则能够识别细粒度的模式，增强模型的表达能力。通过融合这两种特征，能够避免全局特征过度平滑而导致重要局部信息的丢失。本节将介绍的如何对全局特征和局部特征进行融合，并最终将其应用于漏洞检测任务。

特征融合是将来自不同源或层次的特征合并到一起的过程，以充分利用各种类型的特征信息。对不同源的特征直接拼接或者加权求和无法利用它们之间的相互关系，为了弥补这一不足，本文使用交叉注意力对全局特征和局部特征进行融合。交叉注意力机制的核心思想在于通过查询（Query）和键（Key）之间的相似度计算，基于此相似度对不同的值（Value）进行加权，实现特征的有效融合。

在特征融合的过程中，首先将全局特征作为查询向量，局部特征作为键向量和值向量，使得模型能够根据局部特征的内容来调整全局特征表示。具体步骤如下：先计算查询向量和键向量之间的相似度，再通过 softmax 操作对相似度进行归一化得到注意力权重矩阵，如公式(12)所示：

$$AttentionWeights = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) \quad (12)$$

其中 Q 表示全局特征， K 表示局部特征， d_k 表示局部特征的维度。接下来，基于计算得到的注意力权重对值向量进行加权求和得到特征表示 *Feature*，如公式(13)所示：

$$Feature = AttentionWeights \cdot V \quad (13)$$

其中 V 表示局部特征。然后，交换全局特征和局部特征的角色，将局部特征作为查询向量，全局特征作为键向量和值向量，同样经过计算得到相应的 *AttentionWeights'* 和 *Feature'*，使得模型能够根据全局特征的内容来调整局部特征表示。

最后，将全局特征和局部特征经过交叉注意

力机制处理后得到的两种特征 *Feature* 和 *Feature'* 拼接，生成最终的特征表示，并输入全连接层进行二分类任务，从而完成漏洞检测任务。

4. 实验与结果分析

本文的实验在单个 NVIDIA A100 40GB GPU 上完成。代码主要基于 Python3.10 和 Pytorch2.5.1 进行编写。批次大小设置为 128，学习率设置为 0.0001，训练迭代轮数为 20 轮，优化器采用 AdamW 优化器。

4.1 数据集介绍

对于漏洞检测模型来说，在真实世界数据集上的性能表现比在理想化或人工合成的数据集上更为重要。这些数据集包含了来自实际开发环境中的复杂代码，这些代码不仅具有更多样化的漏洞模式，还可能包含更多的噪声和不规范的编码习惯，模型如果无法在这些复杂的真实场景中取得优秀的表现，就无法在实际应用中有效发挥作用。因此本文选取了真实世界的均衡数据集 Devign^[13] 和非均衡数据集 Reveal^[3] 以验证模型性能的有效性，进一步增强模型在不同数据分布下的鲁棒性。对于非均衡数据集 Reveal，采用过采样^[23]的方法来解决其类别不平衡的问题。两个数据集的具体构成如表 1 所示。

表 1 数据集的具体构成

Table 1 Detailed composition of datasets

	漏洞样本	非漏洞样本	总计	来源
Devign	12460	14858	27318	QEMU、FFmpeg
Reveal	2098	20050	22148	Chromium、Debian

4.2 评估指标

为了评估漏洞检测模型的实际效果，本文选用准确率（Accuracy）、精确率（Precision）、召回率（Recall）、F1 分数（F1-Score）作为评估指标。

准确率表示模型预测正确的样本占总样本的

比例；精确率表示模型预测为“有漏洞”的代码样本中，实际存在漏洞的比例；召回率表示实际存在漏洞的样本中，模型成果检测出的比例，F1 分数用于平衡精确率和召回率的重要性。上述各指标的定义如式 (14)~(17) 所示。

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (14)$$

$$Precision = \frac{TP}{TP + FP} \quad (15)$$

$$Recall = \frac{TP}{TP + FN} \quad (16)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (17)$$

其中， TP 表示实际为有漏洞并且预测为有漏洞的样本数， FP 表示实际没有漏洞但被预测为有漏洞的样本数， TN 表示实际没有漏洞且预测为无漏洞的样本数， FN 表示实际有漏洞但被预测为无漏洞的样本数。

4.3 实验结果分析

为了验证说明本文提出的漏洞检测模型的有效性，本文选取多个具有代表性的基线模型作为对比对象，并在对应的数据集上选取相同的评估指标进行测试。实验结果如表 2 所示。选取的基线模型大致可以分为以下两类：

- 基于 Token 序列的模型：CodeBERT、ContraBERT 和 TextCNN。

- 基于图神经网络的模型：GraphCodeBERT、Devign、ReGVD 和 DGVD。

由实验结果可以看出，本文所提出的 MC-CLVD 模型在评估指标上取得了较为显著的性能提升，尤其是在召回率和 F1 分数上相比于其他基线模型有明显优势，体现了在减少漏报方面的强大能力。

基于 Token 序列的模型在处理代码的局部信息方面有明显的优势，能够较为全面的捕捉代码的语义和潜在漏洞。其中 TextCNN 在召回率和 F1 分数上较为均衡，说明卷积神经网络在提取代码局部特征方面有一定优势。然而，由于其对全局信息的建模能力有限，整体性能与本文模型仍然存在差距。

基于图神经网络的模型在某些指标表现相对较好，尤其是在代码的语义理解和长距离依赖捕捉上有独特优势。其中 DGVD 将代码的局部特征提取作为控制流图的嵌入，以此来更好地捕捉代码语义，但由于其对代码结构建模较为依赖，在更复杂场景下的泛化能力有限，综合性能未能超过本文模型。

表 2 MC-CLVD 与其他模型的性能对比结果

Table 2 Performance comparison results of MC-CLVD and other models

	Devign				Reveal			
	ACC(%)	P(%)	Recall(%)	F1(%)	ACC(%)	P(%)	Recall(%)	F1(%)
CodeBERT	61.94	61.13	48.61	54.15	88.64	38.26	38.13	38.19
ContraBERT	63.26	59.81	55.62	57.64	90.90	60.22	23.55	33.86
TextCNN	60.38	59.03	57.72	58.37	85.43	26.32	20.33	22.94
GraphCodeBERT	62.61	63.53	46.77	53.87	89.25	41.67	41.81	41.74
Devign	57.66	56.96	56.25	56.60	65.47	17.38	18.09	17.72
ReGVD	62.15	61.72	46.13	52.80	91.16	53.33	25.33	34.35
DGVD	64.23	61.45	59.33	60.37	-	-	-	-
MC-CLVD	64.86	58.24	67.57	62.56	90.67	49.60	59.05	53.91

综上所述，MC-CLVD 在真实世界复杂代码场景中，面对复杂或者未知漏洞时的优越性主要体现在以下几个方面：

- 全局特征与局部特征的融合：代码结构的复杂性往表现为全局逻辑和局部细节之间的复杂交织。复杂代码中往往涉及不同层级的逻辑判断、函数调用及模块间的交互，传统方法可能难以同时处理这些多层次的信息。MC-CLVD 通过 BiGRU 和 TextCNN 的结合，有效地整合了源代码语义的全局特征和局部特征，从而在复杂代码中既能捕捉到长距离依赖的语义关系，也能够精确地抓取代码的局部模式特征。当全局特征和局部特征相辅相成时，模型能够更加全面地理解代码的结构和潜在的漏洞所在，特别是在代码结构复杂且漏洞并非显示呈现时。

- 特征的动态交互：MC-CLVD 通过引入交叉注意力机制，使得模型能够自适应地聚焦于代码中的关键区域，动态调整对全局特征和局部特征的关注度。在复杂代码或者存在未知漏洞的情况下，潜在漏洞可能隐藏在某些关键区域，这些区域的特征信息并非静态，而是随着代码结构的变化而变化。模型根据输入特征的不同层次，灵活地分配注意力，从而避免了过度依赖固定规则或者模式，增强了对新型、未知漏洞地检测能

力和适应性。这种动态的特征交互模式使得模型在面对多样化的漏洞时展现出更强的泛化能力。

- 对比学习优化特征空间：通过对比学习，MC-CLVD 有效优化了特征空间，将相似样本聚合，拉远不相关样本的距离，提升了漏洞代码与非漏洞代码之间的区分度。同时，通过学习不同漏洞代码的特征，挖掘不同类型漏洞的潜在关联，模型能够识别各种漏洞的共性模式和底层特征，在特征空间中生成更为普适的表示。这使得模型能够自适应地检测未知漏洞，无论其表现形式如何变化，减少了对固定漏洞规则的依赖，显著增强了模型对未知漏洞的检测能力。

4.4 消融实验

为了评估 MC-CLVD 中各个模块对整体性能的贡献程度，设置了如下对照实验，具体实验结果如表 3 所示：

- (1) 仅有全局特征提取模块：BiGRU-VD；
- (2) 仅有局部特征提取模块：TextCNN-VD；
- (3) 双通道特征提取：MC-VD-NO；
- (4) 双通道特征提取+通道注意力模块：MC-VD；
- (5) 双通道特征提取+通道注意力模块+对比学习方法：MC-CLVD。

表 3 消融实验结果

Table 3 Results of ablation experiment

	Devign				Reveal			
	ACC(%)	P(%)	Recall(%)	F1(%)	ACC(%)	P(%)	Recall(%)	F1(%)
BiGRU-VD	60.72	54.37	59.73	56.92	87.99	39.18	54.29	45.51
TextCNN-VD	61.49	54.83	64.53	59.28	88.34	40.35	54.76	46.46
MC-VD-NO	62.66	58.46	64.70	61.42	89.84	45.95	56.67	50.75
MC-VD	63.43	56.66	67.40	61.56	89.93	46.33	57.14	51.17
MC-CLVD	64.86	58.24	67.57	62.56	90.67	49.60	59.05	53.91

实验结果表明，仅包含全局特征提取模块的 BiGRU-VD 和仅包含局部特征提取模块的 TextCNN-VD 表现相对较差，尤其是在 F1 分数这

项指标上。这表明，单独使用全局特征或者局部特征不足以优化模型性能。全局特征可能无法捕捉到代码中的细节信息，而局部特征则可能缺少

对整体结构的感知，均具有一定的局限性。

MC-VD-NO 使用不同通道并列提取了全局特征和局部特征并进行特征融合，显著提高了准确率和精确率，这表明通过同时使用全局特征和局部特征的组合，模型能够更全面的捕捉源代码语义特征。

MC-VD 在提取局部特征时加入了通道注意力机制，进一步提升了模型性能。MC-CLVD 在所有指标上均表现优异，尤其是在召回率和 F1 分数上，证明了引入对比学习能够有效地优化模型的学习过程。

此外，对于不同类型的漏洞来说，其对特征提取方法的依赖性也存在差异。具体而言，缓冲区溢出漏洞通常涉及内存边界的溢出，属于局部数据流失控的情况，因此更加依赖于局部特征的提取，如缓冲区的使用模式、数据流动和边界检测等细节。

相比之下，释放后使用漏洞和注入攻击大多与函数调用序列、内存管理和控制流相关。释放后使用漏洞通常发生在内存释放后，程序仍然试图访问已经释放的内存区域，依赖于程序中的内存管理错误；注入攻击通常利用了程序对外部输入的处理不当，同样也需要追踪整个程序的执行流程。这类漏洞的检测需要模型能够捕捉到程序的全局执行流程和内存变化，而非局部的代码片段或单一功能模块，因此对全局特征的依赖性较强。

综上所述，不同类型的漏洞对特征提取方法的依赖性差异要求模型设计时需要兼顾全局特征和局部特征的有效结合。结合两者的优势并引入对比学习策略，能很大程度上提升模型漏洞检测的准确性和泛化能力。

结束语 基于深度学习的源代码漏洞检测是软件安全领域的重要研究内容。本文设计并实现了 MC-CLVD 漏洞检测模型，并通过实验验证了其在源代码漏洞检测中的卓越性能。通过引入双通道特征提取和对比学习方法，模型能够更全面

地捕捉漏洞特征，并准确识别不同样本之间的相似性与差异性。消融实验进一步揭示了各个部分对模型性能的贡献，为后续的模型优化提供了理论依据。然而，本文的研究仍然存在一定的局限性，例如在处理极为复杂或未知的漏洞时，模型的效果可能仍有提升空间。未来的研究可以进一步探索更高效的特征提取方法、对比学习机制的优化以及更广泛的数据集应用，以进一步提升漏洞检测模型的准确性和适用性。

参考文献

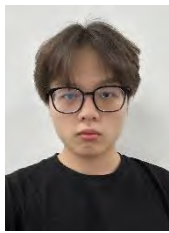
- [1] Skybox Security. Vulnerability & Threat Trends Report 2023[EB/OL]. [2024-11-18]. <https://www.skyboxsecurity.com/resources/report/vulnerability-threat-trends-report-2023/>.
- [2] SU X H, ZHENG W L, JIANG Y, et al. Research and progress on learning-base source code vulnerability detection[J]. Journal of Computers, 2024, 47(02): 337-374.
苏小红, 郑伟宁, 蒋远, 等. 基于学习的源代码漏洞检测研究与进展[J]. 计算机学报, 2024, 47(02): 337-374.
- [3] Chakraborty S, Krishna R, Ding Y, et al. Deep learning based vulnerability detection: Are we there yet?[J]. IEEE Transactions on Software Engineering, 2021, 48(9): 3280-3296.
- [4] Chen T, Kornblith S, Norouzi M, et al. A simple framework for contrastive learning of visual representations[C]//International conference on machine learning. PmLR, 2020: 1597-1607.
- [5] Lin H, Cheng X, Wu X, et al. Cat: Cross attention in vision transformer[C]//2022 IEEE international conference on multimedia and expo (ICME). IEEE, 2022: 1-6.
- [6] Feng Z, Guo D, Tang D, et al. Codebert: A pre-trained model for programming and natural languages[J]. arXiv preprint

-
- arXiv:2002.08155, 2020.
- [7] Liu Y. Roberta: A robustly optimized bert pretraining approach[J]. arXiv preprint arXiv:1907.11692, 2019, 364.
- [8] Guo D, Ren S, Lu S, et al. Graphcodebert: Pre-training code representations with data flow[J]. arXiv preprint arXiv:2009.08366, 2020.
- [9] Russell R, Kim L, Hamilton L, et al. Automated vulnerability detection in source code using deep representation learning[C]//2018 17th IEEE international conference on machine learning and applications (ICMLA).IEEE, 2018: 757-762.
- [10] Li Z, Zou D, Xu S, et al. Vuldeepecker: A deep learning-based system for vulnerability detection[J]. arXiv preprint arXiv:1801.01681, 2018.
- [11] Hanif H, Maffei S. Vulberta: Simplified source code pre-training for vulnerability detection[C]//2022 International joint conference on neural networks (IJCNN). IEEE, 2022: 1-8.
- [12] Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.
- [13] Zhou Y, Liu S, Siow J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. Advances in neural information processing systems, 2019, 32.
- [14] Nguyen V A, Nguyen D Q, Nguyen V, et al. ReGVD: Revisiting graph neural networks for vulnerability detection[C]//Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. 2022: 178-182.
- [15] Ling M, Tang M, Bian D, et al. A dual graph neural networks model using sequence embedding as graph nodes for vulnerability detection[J]. Information and Software Technology, 2025, 177: 107581.
- [16] Jain P, Jain A, Zhang T, et al. Contrastive code representation learning[J]. arXiv preprint arXiv:2007.04973, 2020.
- [17] Neelakantan A, Xu T, Puri R, et al. Text and code embeddings by contrastive pre-training[J]. arXiv preprint arXiv:2201.10005, 2022.
- [18] Liu S, Wu B, Xie X, et al. Contrabert: Enhancing code pre-trained models via contrastive learning[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 2476-2487.
- [19] Chen Y, Sun Z, Gong Z, et al. Improving Smart Contract Security with Contrastive Learning-based Vulnerability Detection[C]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-11.
- [20] LLVM Team. libclang: C Interface to Clang[EB/OL]. [2024-11-18]. https://clang.llvm.org/doxygen/group__CINDEX.html.
- [21] Gage P. A new algorithm for data compression[J]. The C Users Journal, 1994, 12(2): 23-38.
- [22] Hu J, Shen L, Sun G. Squeeze-and-excitation networks[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 7132-7141.
- [23] Chawla N V, Bowyer K W, Hall L O, et al.

SMOTE: synthetic minority over-sampling technique[J]. Journal of artificial intelligence research, 2002, 16: 321-357.



SONG Jian-Hua, born in 1973, Ph.D, professor, postgraduate supervisor, is a member of CCF(27785M). Her main research interests include Network and Information Security.



HE Jia-Wei, born in 2001, is a master's student with a main research focus on source code vulnerability detection.